

Common workflows

Step-by-step guides for exploring codebases, fixing bugs, refactoring, testing, and other everyday tasks with Claude Code.

DESIGNED HTML FIRST · PRINTED TO PDF WITH CHROME

SOURCE	examples/common-workflows/common-workflows.md
LINES	528
SECTIONS	7
CODE BLOCKS	48

Section map

- 01** Prompt recipes
- 02** Resume previous conversations
- 03** Run parallel sessions with worktrees
- 04** Plan before editing
- 05** Delegate research to subagents
- 06** Pipe Claude into scripts
- 07** Next steps

Documentation Index

Fetch the complete documentation index at:
<https://code.claude.com/docs/llms.txt>

Use this file to discover all available pages before exploring further.

Step-by-step guides for exploring codebases, fixing bugs, refactoring, testing, and other everyday tasks with Claude Code.

This page collects short recipes for everyday development. For higher-level guidance on prompting and context management, see [Best practices](#).

This page covers:

- [Prompt recipes](#) for exploring code, fixing bugs, refactoring, testing, PRs, and documentation
- [Resume previous conversations](#) so a task can span multiple sittings
- [Run parallel sessions with worktrees](#) so concurrent edits don't collide
- [Plan before editing](#) to review changes before they touch disk
- [Delegate research to subagents](#) to keep your main context clean
- [Pipe Claude into scripts](#) for CI and batch processing

Prompt recipes

These are prompt patterns for everyday tasks like exploring unfamiliar code, debugging, refactoring, writing tests, and creating PRs. Each works in any Claude Code surface; adapt the wording to your project.

Understand new codebases

For configuring Claude Code in a monorepo or large codebase, see [Monorepos and large repos](#).

Get a quick codebase overview

Suppose you've just joined a new project and need to understand its structure quickly.

01

Navigate to the project root directory

```
cd /path/to/project
```

02

Start Claude Code

```
claude
```

03

Ask for a high-level overview

```
give me an overview of this codebase
```

04

Dive deeper into specific components

```
explain the main architecture patterns used here
```

```
what are the key data models?
```

```
how is authentication handled?
```

Tips:

- Start with broad questions, then narrow down to specific areas
- Ask about coding conventions and patterns used in the project
- Request a glossary of project-specific terms

Find relevant code

Suppose you need to locate code related to a specific feature or functionality.

01

Ask Claude to find relevant files

```
find the files that handle user authentication
```

02

Get context on how components interact

```
how do these authentication files work together?
```

03

Understand the execution flow

```
trace the login process from front-end to database
```

Tips:

- Be specific about what you're looking for
- Use domain language from the project

- Install a [code intelligence plugin](#) for your language to give Claude precise "go to definition" and "find references" navigation

Fix bugs efficiently

Suppose you've encountered an error message and need to find and fix its source.

01

Share the error with Claude

```
I'm seeing an error when I run npm test
```

02

Ask for fix recommendations

```
suggest a few ways to fix the @ts-ignore in user.ts
```

03

Apply the fix

```
update user.ts to add the null check you suggested
```

Tips:

- Tell Claude the command to reproduce the issue and get a stack trace
- Mention any steps to reproduce the error
- Let Claude know if the error is intermittent or consistent

Refactor code

Suppose you need to update old code to use modern patterns and practices.

01

Identify legacy code for refactoring

```
find deprecated API usage in our codebase
```

02

Get refactoring recommendations

```
suggest how to refactor utils.js to use modern JavaScript features
```

03

Apply the changes safely

```
refactor utils.js to use ES2024 features while maintaining the same behavior
```

04

Verify the refactoring

```
run tests for the refactored code
```

Tips:

- Ask Claude to explain the benefits of the modern approach
- Request that changes maintain backward compatibility when needed
- Do refactoring in small, testable increments

Work with tests

Suppose you need to add tests for uncovered code.

01

Identify untested code

```
find functions in NotificationsService.swift that are not covered by tests
```

02

Generate test scaffolding

```
add tests for the notification service
```

03

Add meaningful test cases

```
add test cases for edge conditions in the notification service
```

04

Run and verify tests

```
run the new tests and fix any failures
```

Claude can generate tests that follow your project's existing patterns and conventions. When asking for tests, be specific about what behavior you want to verify. Claude examines your existing test files to match the style, frameworks, and assertion patterns already in use.

For comprehensive coverage, ask Claude to identify edge cases you might have missed. Claude can analyze your code paths and suggest tests for error conditions, boundary values, and unexpected inputs that are easy to overlook.

Create pull requests

You can create pull requests by asking Claude directly ("create a pr for my changes"), or guide Claude through it step-by-step:

01

Summarize your changes

```
summarize the changes I've made to the authentication module
```

02

Generate a pull request

```
create a pr
```

03

Review and refine

```
enhance the PR description with more context about the security improvements
```

When you create a PR using `gh pr create`, the session is automatically linked to that PR. To return to it later, run `claude --from-pr <number>` or paste the PR URL into the `/resume` picker search.

Review Claude's generated PR before submitting and ask Claude to highlight potential risks or considerations.

Handle documentation

Suppose you need to add or update documentation for your code.

01

Identify undocumented code

```
find functions without proper JSDoc comments in the auth module
```

02

Generate documentation

```
add JSDoc comments to the undocumented functions in auth.js
```

03

Review and enhance

```
improve the generated documentation with more context and examples
```

04

Verify documentation

```
check if the documentation follows our project standards
```

Tips:

- Specify the documentation style you want (JSDoc, docstrings, etc.)
- Ask for examples in the documentation
- Request documentation for public APIs, interfaces, and complex logic

Work in notes and non-code folders

Claude Code works in any directory. Run it inside a notes vault, a documentation folder, or any collection of markdown files to search, edit, and reorganize content the same way you would code.

The `.claude/` directory and `CLAUDE.md` sit alongside other tools' config directories without conflict. Claude reads files fresh on each tool call, so it sees edits you make in another application the next time it reads that file.

Work with images

Suppose you need to work with images in your codebase, and you want Claude's help analyzing image content.

01

Add an image to the conversation

PDF-friendly web edition - source converted from markdown

You can use any of these methods:

1. Drag and drop an image into the Claude Code window
2. Copy an image and paste it into the CLI with ctrl+v (Do not use cmd+v)
3. Provide an image path to Claude. E.g., "Analyze this image: /path/to/your/image.png"

02

Ask Claude to analyze the image

What does this image show?

Describe the UI elements in this screenshot

Are there any problematic elements in this diagram?

03

Use images for context

Here's a screenshot of the error. What's causing it?

This is our current database schema. How should we modify it for the new feature?

04

Get code suggestions from visual content

Generate CSS to match this design mockup

What HTML structure would recreate this component?

Tips:

- Use images when text descriptions would be unclear or cumbersome
- Include screenshots of errors, UI designs, or diagrams for better context
- You can work with multiple images in a conversation
- Image analysis works with diagrams, screenshots, mockups, and more
- When Claude references images (for example, `[Image #1]`), `Cmd+Click` (Mac) or `Ctrl+Click` (Windows/Linux) the link to open the image in your default viewer

Reference files and directories

Use `@` to quickly include files or directories without waiting for Claude to read them.

01

Reference a single file

```
Explain the logic in @src/utils/auth.js
```

This includes the full content of the file in the conversation.

02

Reference a directory

```
What's the structure of @src/components?
```

This provides a directory listing with file information.

03

Reference MCP resources

```
Show me the data from @github:repos/owner/repo/issues
```

This fetches data from connected MCP servers using the format `@server:resource`. See [MCP resources](#) for details.

Tips:

- File paths can be relative or absolute
- @ file references add `CLAUDE.md` in the file's directory and parent directories to context
- Directory references show file listings, not contents
- You can reference multiple files in a single message (for example, "@file1.js and @file2.js")

Run Claude on a schedule

Suppose you want Claude to handle a task automatically on a recurring basis, like reviewing open PRs every morning, auditing dependencies weekly, or checking for CI failures overnight.

Pick a scheduling option based on where you want the task to run:

Option	Where it runs	Best for
<code>--cron</code>	Anthropic-managed infrastructure	Tasks that should run even when your computer is off. Can also trigger on API calls or GitHub events in addition to a schedule. Configure at claude.ai/code/routines .
<code>--loop</code>	Your machine, via the desktop app	Tasks that need direct access to local files, tools, or uncommitted changes.
<code>--github-actions</code>	Your CI pipeline	Tasks tied to repo events like opened PRs, or cron schedules that should live alongside your workflow config.
<code>--poll</code>	The current CLI session	Quick polling while a session is open. Tasks stop when you start a new conversation; <code>--resume</code> and <code>--continue</code> restore unexpired ones.

When writing prompts for scheduled tasks, be explicit about what success looks like and what to do with results. The task runs autonomously, so it can't ask clarifying questions. For example: "Review open PRs labeled `needs-review`, leave inline comments on any issues, and post a summary in the `#eng-reviews` Slack channel."

Ask Claude about its capabilities

Claude has built-in access to its documentation and can answer questions about its own features and limitations.

Example questions

can Claude Code create pull requests?

how does Claude Code handle permissions?

what skills are available?

how do I use MCP with Claude Code?

how do I configure Claude Code for Amazon Bedrock?

what are the limitations of Claude Code?

<Note> Claude provides documentation-based answers to these questions. For hands-on demonstrations, run `/powerup` for interactive lessons with animated demos, or refer to the specific workflow sections above. </Note>

Tips:

- Claude always has access to the latest Claude Code documentation, regardless of the version you're using
- Ask specific questions to get detailed answers
- Claude can explain complex features like MCP integration, enterprise configurations, and advanced workflows

Resume previous conversations

When a task spans multiple sittings, pick up where you left off instead of re-explaining context. Claude Code saves every conversation locally.

```
claude --continue
```

This resumes the most recent session in the current directory; if there isn't one yet, it prints `No conversation found to continue` and exits. Use `claude --resume` to choose from a list, or `/resume` from inside a running session. See [Manage sessions](#) for naming, branching, and the full picker reference.

Run parallel sessions with worktrees

Work on a feature in one terminal while Claude fixes a bug in another, without the edits colliding. Each worktree is a separate checkout on its own branch.

```
claude --worktree feature-auth
```

Run the same command with a different name in a second terminal to start an isolated parallel session. See [Worktrees](#) for cleanup, `.worktreeinclude`, and non-git VCS support. To monitor parallel sessions from one screen instead of separate terminals, see [background agents](#).

Plan before editing

For changes you want to review before they touch disk, switch to plan mode. Claude reads files and proposes a plan but makes no edits until you approve.

```
claude --permission-mode plan
```

You can also press `Shift+Tab` mid-session to toggle into plan mode. See [Plan mode](#) for the approval flow and editing the plan in your text editor.

Delegate research to subagents

Exploring a large codebase fills your context with file reads. Delegate the exploration so only the findings come back.

```
use a subagent to investigate how our auth system handles token refresh
```

The subagent reads files in its own context window and reports a summary. See [Subagents](#) for defining custom agents with their own tools and prompts.

Pipe Claude into scripts

Run Claude non-interactively for CI, pre-commit hooks, or batch processing. Stdin and stdout work like any Unix tool.

```
git log --oneline -20 | claude -p "summarize these recent commits"
```

See [Non-interactive mode](#) for output formats, permission flags, and fan-out patterns.

Next steps

<CardGroup cols={2}> <Card title="Best practices" icon="lightbulb" href="/en/best-practices"> Patterns for getting the most out of Claude Code </Card>

<Card title="Manage sessions" icon="rotate-left" href="/en/sessions"> Resume, name, and branch conversations </Card>

<Card title="Worktrees" icon="code-branch" href="/en/worktrees"> Run isolated parallel sessions </Card>

<Card title="Extend Claude Code" icon="puzzle-piece" href="/en/features-overview"> Add skills, hooks, MCP, subagents, and plugins </Card> </CardGroup>